BASES DE DATOS I

Práctica 1: MotoGP

Grupo Prac1_1

Ni, Aibo
Ye, Sergio

Curso 2025–26
MATIC (ETSISI–UPM)

Índice

1. Introducción **2**

2. Auditoría de datos **2**

3. Modelo conceptual **3**

4. Modelo lógico relacional **4**

5. Dependencias funcionales y formas normales **5**

6. Implementación y migración **6**

7. Consultas y explotación de resultados **9**

8. Conclusiones **16**

1. Introducción

En esta práctica partimos de una base de datos heredada con casi 30 000 registros almacenados en un csv. Dichos registros mezclaban información de pilotos, equipos, circuitos y resultados de carrera sin la presencia de claves primarias ni foráneas. Nuestro objetivo fue auditar estos datos, normalizarlos para eliminar redundancias y proponer un esquema relacional robusto. En este informe documentamos todas las etapas que hemos seguido: la auditoría de datos, el diseño de los modelos conceptual y lógico, la migración de los registros a MySQL y la explotación de la base mediante consultas analíticas.

2. Auditoría de datos

Durante la auditoría analizamos el fichero `moto_results.csv` mediante Python y sus respectivas librerías. Identificamos 29 931 filas y 17 columnas que recogen, entre otros campos, la temporada, el orden en el calendario (`sequence`), la categoría (MotoGP, Moto2, Moto3, etc.), el nombre y apellidos del piloto, el dorsal, la nacionalidad, el nombre del equipo, la marca de la moto, la posición alcanzada, los puntos, la velocidad media, el tiempo registrado y el circuito donde se celebra la carrera.

Exploración inicial y limpieza

- Convertimos la columna `date` al tipo fecha para facilitar filtrados. La tabla abarca carreras entre 2000 y 2021, y las categorías se distribuyen así: MotoGP (7023 registros), Moto2 (6755), 125cc (6013), Moto3 (5518), 250cc (3856), 500cc (502) y MotoE (264).

- Se detectaron inconsistencias de formato en la columna `circuit_name` que generaban duplicados lógicos del mismo circuito. Es por ello que aplicamos una normalización de formato en los nombres mediante `strip()` y `title()`. Además, corregimos el país del circuito MotorLand Aragón, que figuraba como `PT` a `ES`.

- Analizamos los valores nulos: `rider_number` y `speed` eran las columnas con más ausencias con 5.127 y 992 valores nulos respectivamente. Observamos que las filas con dorsal nulo coincidían con el equipo '?'. Esta coincidencia indica que los registros históricos sin dorsal conocido corresponden sistemáticamente a participaciones sin equipo identificado en la fuente original. Antes de la migración, el token '?' fue sustituido por None.

- El campo `position` contiene 5.475 valores no positivos, correspondientes al 18,29 % del total. Se verificó que la integridad de puntos se mantiene en todos estos casos, es decir, confirmamos que ningún registro con posición no positiva tiene puntos asignados.

- Comprobamos que no existían claves naturales: el dorsal se repite entre temporadas y la combinación de nombre y apellidos tampoco es única. Encontramos 14 casos donde un dorsal se asociaba a pilotos distintos en el mismo año y categoría, y 329 casos donde un piloto aparecía con más de un equipo en la misma temporada. Por tanto, generamos identificadores artificiales.

3. Modelo conceptual

A partir del análisis del dominio identificamos las entidades principales de MotoGP: *Piloto*, *Equipo*, *Circuito*, *Carrera* y *Resultado*. A continuación presentamos el diagrama conceptual con notación Martín, que resume gráficamente las entidades, sus claves primarias y las relaciones con cardinalidades.

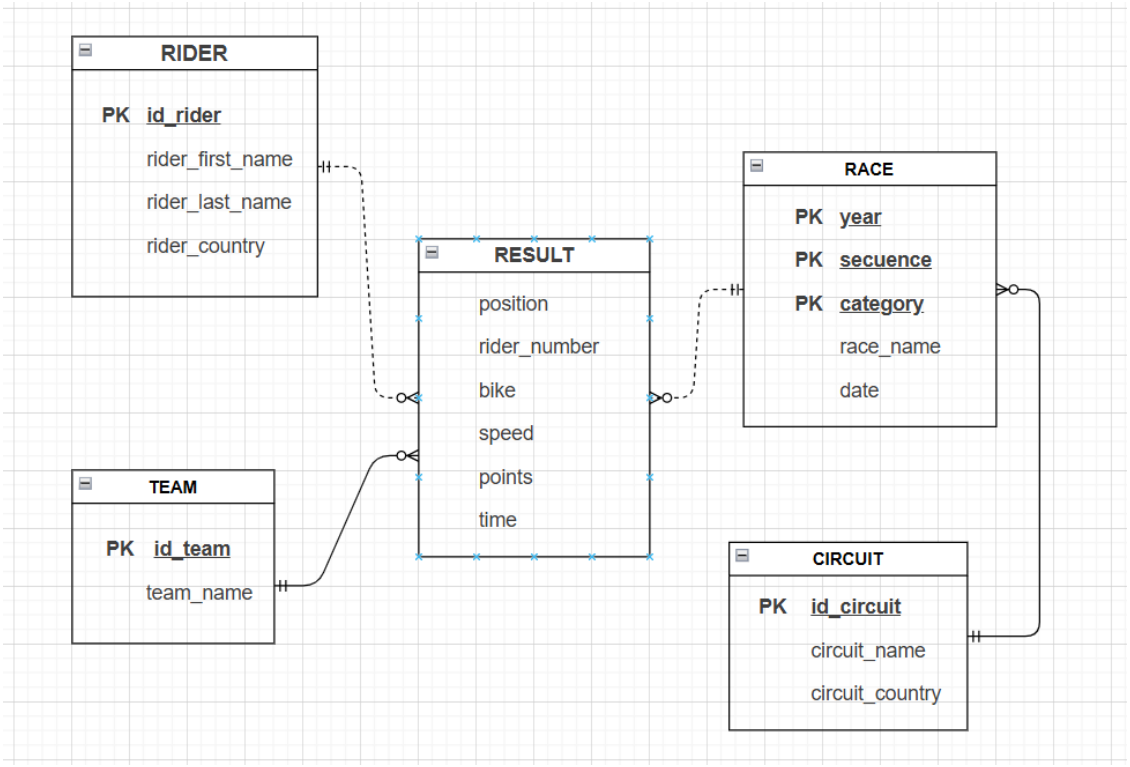

Figura 1: Diagrama conceptual de las entidades y relaciones.

Descripción de entidades y relaciones

RIDER Representamos a cada corredor mediante un identificador artificial `id_rider`, junto con sus atributos de nombre, apellidos y nacionalidad.

TEAM Definimos cada equipo con `id_team` y un nombre.

CIRCUIT Cada uno cuenta con un identificador (`id_circuit`), un nombre y el país donde se ubica.

RACE Se identifica con `(year, sequence, category)` y contiene atributos como el nombre y la fecha.

RESULT Relaciona a un piloto con una carrera y su equipo; recoge posición, dorsal, moto, velocidad, puntos y tiempo.

4. Modelo lógico relacional

Del modelo conceptual derivamos un esquema relacional. En la Figura 2 se muestran las tablas resultantes con sus claves primarias y foráneas.

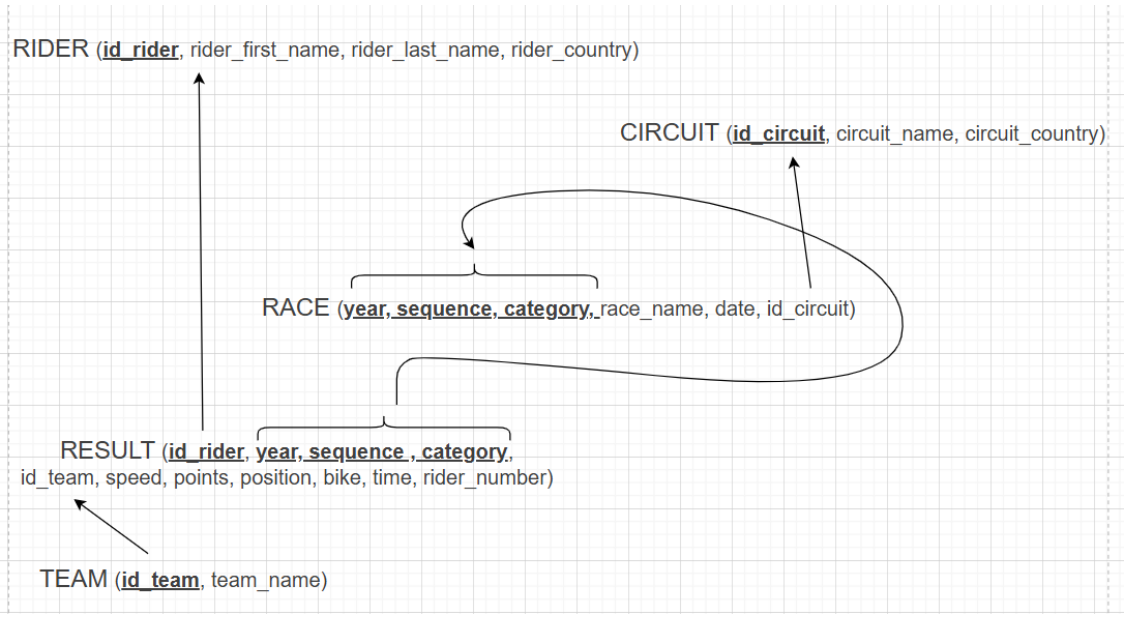

Figura 2: Diagrama lógico–relacional con las tablas normalizadas y sus claves.

Esquema de tablas

- `RIDERS` (id_rider PK, rider_first_name, rider_last_name, rider_country).
- `TEAMS` (id_team PK, team_name).
- `CIRCUITS` (id_circuit PK, circuit_name, circuit_country).
- `RACES` (year PK, sequence PK, category PK, race_name, date, id_circuit).[1]
- `RESULTS` (id_rider FK, year FK, sequence FK, category FK, id_team, bike, position, points, speed, time, rider_number).

[1] El análisis de dependencias funcionales (sección 6) revela una dependencia parcial en esta entidad, lo que motiva su descomposición en GRAND_PRIX y RACE para alcanzar la Segunda Forma Normal (2FN). El esquema definitivo se presenta en dicha sección.

5. Dependencias funcionales y formas normales

Una vez definido el modelo relacional inicial, analizamos las dependencias funcionales presentes en los datos con el objetivo de garantizar un diseño normalizado y evitar redundancias. A partir del análisis exploratorio del dataset original, identificamos varias conclusiones relevantes. En primer lugar, comprobamos que no existe una clave natural para la entidad `RIDER`: ni el nombre ni el dorsal identifican de manera única a un piloto, ya que ambos pueden repetirse. Por este motivo, fue necesario introducir un identificador artificial `id_rider`.

Por otro lado, verificamos que la tripleta `(year, sequence, category)` identifica de forma única cada carrera, por lo que se establece como clave primaria de la entidad `RACE`. En cuanto a las dependencias funcionales, observamos que ciertos atributos no dependen completamente de la clave compuesta de `RACE`. En concreto, atributos como `race_name`, `date` e `id_circuit` dependen únicamente de `(year, sequence)`, ya que describen el evento del gran premio y no la carrera individual de cada categoría. Esto se puede expresar como:

$$(year, sequence) \rightarrow race_name, date, id_circuit$$

Dado que la clave original es `(year, sequence, category)`, esta situación constituye una **dependencia parcial**, lo que implica que la tabla no cumple la Segunda Forma Normal (2FN).

Para resolver este problema, separamos la información en dos entidades:

- **GRAND_PRIX (year, sequence, race_name, date, id_circuit)**.
- **RACE (year, sequence, category)**.

Esta descomposición elimina la dependencia parcial y garantiza que todos los atributos dependan completamente de su clave primaria.

Además, el análisis de los datos muestra que la relación entre equipo y moto no es funcional. Es decir, un mismo equipo puede utilizar diferentes motos a lo largo del tiempo, e incluso en una misma temporada. Por tanto, no se cumple la dependencia:

$$team \rightarrow bike$$

Esto implica que el atributo `bike` no debe almacenarse en la entidad `TEAM`, sino en la entidad `RESULT`, donde queda correctamente asociado a la participación concreta de un piloto en una carrera.

Asimismo, comprobamos que el atributo `race_name` no es un identificador fiable, ya que un mismo nombre puede corresponder a diferentes circuitos en distintos años. Por ello, no se utiliza como clave y se mantiene como atributo descriptivo.

En relación con la Primera Forma Normal (1FN), el modelo la cumple completamente, ya que todos los atributos contienen valores atómicos y no existen estructuras repetidas.

Finalmente, verificamos que el esquema resultante cumple la Tercera Forma Normal (3FN), ya que no existen dependencias transitivas entre atributos no clave. Todos los atributos dependen directamente de la clave primaria correspondiente, lo que garantiza un diseño consistente y libre de redundancias. Cabe mencionar que para las entidades `CIRCUIT`

y `GRAND_PRIX` se detectaron dependencias aparentes (`circuit_name → circuit_country; date → race_name`), pero se determinó que corresponden a claves candidatas alternativas, no a dependencias transitivas genuinas, por lo que no constituyen violación de la 3FN.

En conjunto, el proceso de normalización nos permite obtener un modelo más claro, estructurado y adecuado para su explotación mediante consultas complejas.

6. Implementación y migración

El fichero `motogp.sql` crea el esquema motogp con codificación utf8 y cotejamiento `utf8_spanish2_ci`, define las seis tablas en orden de dependencia referencial, e incluye un bloque de vaciado seguro para facilitar la recarga.

La migración de datos se basa en un proceso de dos fases. En primer lugar, utilizamos SQLite como entorno intermedio de trabajo. Se trata de un sistema gestor de bases de datos ligero que no requiere servidor, lo que facilita la integración con Python y permite validar el esquema de forma rápida.

Partiendo del fichero original `moto_results.csv`, cargamos los datos mediante la librería `pandas` y aplicamos las transformaciones necesarias detectadas durante la auditoría: normalización de nombres de circuito, corrección de valores inconsistentes y preparación de los datos para su inserción en un modelo relacional. Además, generamos identificadores artificiales para las entidades principales (pilotos, equipos y circuitos), ya que el dataset original no disponía de claves primarias claras.

Una vez preparados los datos, creamos las tablas normalizadas en SQLite mediante `sqlite3` e insertamos los registros, obteniendo al final el archivo `motogp.db`. Esta fase nos permitió comprobar que el modelo era consistente y que no se producían violaciones de integridad.

En segundo lugar, tras validar el esquema, generamos un script SQL compatible con MySQL, obteniendo el archivo final `motogp_mysql.sql` con el que trabajaremos en MySQL Workbench.

Durante la migración detectamos un problema relevante relacionado con la codificación de los países. En la tabla `riders`, la nacionalidad de los pilotos se encontraba en formato ISO de tres letras (por ejemplo, `SPA`, `ITA`), mientras que en la tabla `circuits` el país se almacenaba con códigos de dos letras (por ejemplo, `ES`, `IT`). Esta diferencia impedía realizar comparaciones directas entre ambos atributos, lo cual afectaba especialmente a la consulta 6, donde se requiere comparar la nacionalidad del piloto con el país del circuito.

Para resolver este problema, decidimos unificar ambos formatos utilizando el estándar de tres letras. Esta transformación se implementó mediante una sentencia `UPDATE` con una expresión `CASE`, que sustituye cada código de dos letras por su equivalente correspondiente.

```sql
UPDATE circuits
SET country = CASE country
    WHEN 'AR' THEN 'ARG'
    WHEN 'AT' THEN 'AUT'
    WHEN 'AU' THEN 'AUS'
    WHEN 'BR' THEN 'BRA'
    WHEN 'CN' THEN 'CHN'
    WHEN 'CZ' THEN 'CZE'
    WHEN 'DE' THEN 'GER'
    WHEN 'ES' THEN 'SPA'
    WHEN 'FR' THEN 'FRA'
    WHEN 'GB' THEN 'GBR'
    WHEN 'IT' THEN 'ITA'
    WHEN 'JP' THEN 'JPN'
    WHEN 'MY' THEN 'MAL'
    WHEN 'NL' THEN 'NED'
    WHEN 'PT' THEN 'POR'
    WHEN 'QA' THEN 'QAT'
    WHEN 'TH' THEN 'THA'
    WHEN 'TR' THEN 'TUR'
    WHEN 'US' THEN 'USA'
    WHEN 'ZA' THEN 'RSA'
    ELSE country
END;
```

Gracias a esta corrección, conseguimos que los datos fueran comparables entre tablas, lo que permitió ejecutar correctamente las consultas analíticas sin inconsistencias.

Finalmente, tras aplicar todos los ajustes y completar la migración, obtenemos un esquema relacional consistente y completamente funcional. El diagrama actualizado que se muestra a continuación refleja la estructura final de la base de datos, incluyendo las claves primarias y foráneas, así como las relaciones entre las distintas entidades.

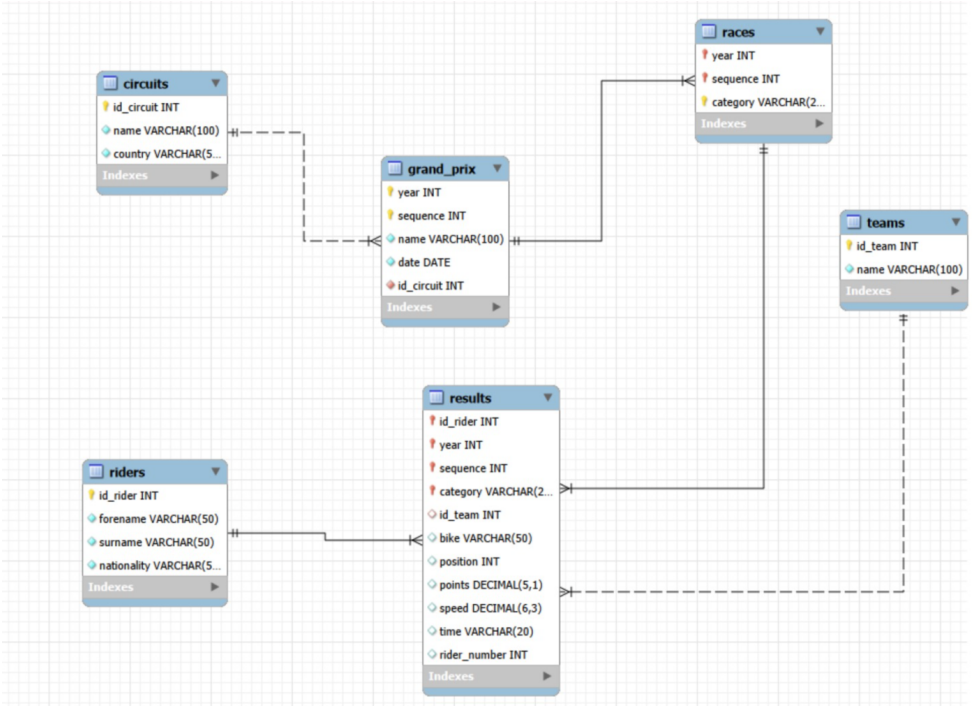

Figura 3: Diagrama E-R actualizado.

Este diagrama no solo representa el resultado final del diseño, sino también las decisiones tomadas durante el proceso de normalización y migración, como la separación de entidades, la definición de claves compuestas y la gestión de atributos dependientes del contexto como `id_team` o `bike`.

7. Consultas y explotación de resultados

Una vez disponible la base de datos normalizada, elaboramos una serie de consultas para extraer la información de interés.

Consulta 1: campeón del mundo más reciente en MotoGP

El objetivo es obtener el piloto o pilotos con mayor suma de puntos en la temporada más reciente de MotoGP. Para ello identificamos el año máximo en la categoría MotoGP y agrupamos los puntos por piloto en ese año. Utilizamos una condición `HAVING` con `ALL` para devolver todos los campeones en caso de empate.

```sql
SELECT ri.forename, ri.surname, SUM(re.points) AS puntos_totales
FROM riders ri
  INNER JOIN results re ON re.id_rider = ri.id_rider
  INNER JOIN races   ra ON ra.year     = re.year
    AND ra.sequence = re.sequence
    AND ra.category = re.category
WHERE ra.category = 'MotoGP'
  AND re.year = (
    SELECT MAX(re2.year) FROM results re2
      INNER JOIN races ra2 ON ra2.year     = re2.year
        AND ra2.sequence  = re2.sequence
        AND ra2.category  = re2.category
    WHERE ra2.category = 'MotoGP'
  )
GROUP BY ri.id_rider, ri.forename, ri.surname
HAVING SUM(re.points) >= ALL (
    SELECT SUM(re2.points) FROM results re2
      INNER JOIN races ra2 ON ra2.year     = re2.year
        AND ra2.sequence  = re2.sequence
        AND ra2.category  = re2.category
    WHERE ra2.category = 'MotoGP'
      AND ra2.year = (
        SELECT MAX(ra3.year) FROM races ra3
        WHERE ra3.category = 'MotoGP'
      )
    GROUP BY re2.id_rider
);
```

Con esta consulta obtenemos el nombre y apellidos del campeón (o campeones) de la última temporada registrada de MotoGP. Utilizar `MAX` y `HAVING` nos permite evitar perder campeones en caso de empate.

Cuadro 1: Resultado Consulta 1: Campeón del mundo MotoGP en el año más reciente (2021)

forename	surname	puntos_totales
Fabio	Quartararo	267.0

Consulta 2: país con más pilotos entre 2010 y 2019

Esta consulta busca el país o países con mayor número de pilotos distintos que hayan competido al menos una vez entre 2010 y 2019, excluyendo la categoría MotoE. Contamos los pilotos distintos por nacionalidad y seleccionamos el máximo.

```sql
SELECT ri.nationality, COUNT(DISTINCT ri.id_rider) AS contador
FROM riders ri
  INNER JOIN results re ON re.id_rider = ri.id_rider
  INNER JOIN races   ra ON ra.year     = re.year
    AND ra.sequence = re.sequence
    AND ra.category = re.category
WHERE ra.category <> 'MotoE'
  AND re.year BETWEEN 2010 AND 2019
GROUP BY ri.nationality
HAVING contador = (
  SELECT MAX(T.contador)
  FROM (
    SELECT COUNT(DISTINCT ri2.id_rider) AS contador
    FROM riders ri2
      INNER JOIN results re2 ON re2.id_rider = ri2.id_rider
      INNER JOIN races   ra2 ON ra2.year     = re2.year
        AND ra2.sequence = re2.sequence
        AND ra2.category = re2.category
    WHERE ra2.category <> 'MotoE'
      AND re2.year BETWEEN 2010 AND 2019
    GROUP BY ri2.nationality
  ) T
);
```

Seleccionamos las nacionalidades que alcanzan el conteo máximo, de este modo, si hay empate entre países, aparecerán todos ellos.

Cuadro 2: Resultado Consulta 2: País con mayor número de pilotos distintos en la década 2010–2019 (excluyendo MotoE)

nationality	contador
ITA	80

Consulta 3: pilotos que han ganado en MotoGP, Moto2 y Moto3

En esta consulta queremos encontrar los pilotos que han ganado al menos una carrera en cada una de las tres categorías principales (MotoGP, Moto2 y Moto3). Contamos las victorias por piloto y determinamos en cuántas categorías distintas ha ganado.

```sql
SELECT ri.forename, ri.surname
FROM riders ri
  INNER JOIN results re ON re.id_rider = ri.id_rider
  INNER JOIN races   ra ON ra.year     = re.year
    AND ra.sequence = re.sequence
    AND ra.category = re.category
WHERE ra.category IN ('MotoGP','Moto2','Moto3')
  AND re.position = 1
GROUP BY ri.id_rider, ri.forename, ri.surname
HAVING COUNT(DISTINCT ra.category) = 3;
```

Esta consulta devuelve los nombres de los pilotos que han logrado al menos una victoria en MotoGP, Moto2 y Moto3, sin importar la temporada.

Cuadro 3: Resultado Consulta 3: Pilotos con al menos una victoria en MotoGP, Moto2 y Moto3

forename	surname
Miguel	Oliveira
Jorge	Martin
Francesco	Bagnaia
Brad	Binder
Maverick	Viñales
Alex	Rins

Consulta 4: campeones en Moto2 y Moto3 pero no en MotoGP

El objetivo es localizar a los pilotos que fueron campeones en Moto2 y Moto3 pero nunca llegaron a serlo en MotoGP. Para cada piloto, año y categoría calculamos la suma total de puntos y la comparamos con la suma máxima de su categoría en ese año. Seleccionamos a los pilotos que aparecen como campeones en Moto2 y Moto3 y que no están en la lista de campeones de MotoGP.

```
SELECT ri.forename, ri.surname
FROM riders ri
WHERE ri.id_rider IN (
    SELECT re.id_rider
    FROM results re
      INNER JOIN races ra ON ra.year     = re.year
        AND ra.sequence = re.sequence
        AND ra.category = re.category
    WHERE ra.category = 'Moto2'
    GROUP BY re.id_rider, ra.year
    HAVING SUM(re.points) = (
      SELECT MAX(puntos_totales) FROM (
        SELECT SUM(re2.points) AS puntos_totales
        FROM results re2
          INNER JOIN races ra2 ON ra2.year     = re2.year
            AND ra2.sequence  = re2.sequence
            AND ra2.category  = re2.category
        WHERE ra2.category = 'Moto2'
          AND ra2.year = ra.year
        GROUP BY re2.id_rider
      ) AS T
    )
)
AND ri.id_rider IN (
    SELECT re.id_rider
    FROM results re
      INNER JOIN races ra ON ra.year     = re.year
        AND ra.sequence = re.sequence
        AND ra.category = re.category
    WHERE ra.category = 'Moto3'
    GROUP BY re.id_rider, ra.year
    HAVING SUM(re.points) = (
      SELECT MAX(puntos_totales) FROM (
        SELECT SUM(re2.points) AS puntos_totales
        FROM results re2
          INNER JOIN races ra2 ON ra2.year     = re2.year
            AND ra2.sequence  = re2.sequence
            AND ra2.category  = re2.category
        WHERE ra2.category = 'Moto3'
          AND ra2.year = ra.year
```

```sql
        GROUP BY re2.id_rider
      ) AS T
    )
)
AND ri.id_rider NOT IN (
    SELECT re.id_rider
    FROM results re
      INNER JOIN races ra ON ra.year     = re.year
        AND ra.sequence = re.sequence
        AND ra.category = re.category
    WHERE ra.category = 'MotoGP'
    GROUP BY re.id_rider, ra.year
    HAVING SUM(re.points) = (
      SELECT MAX(puntos_totales) FROM (
        SELECT SUM(re2.points) AS puntos_totales
        FROM results re2
          INNER JOIN races ra2 ON ra2.year     = re2.year
            AND ra2.sequence  = re2.sequence
            AND ra2.category  = re2.category
        WHERE ra2.category = 'MotoGP'
          AND ra2.year = ra.year
        GROUP BY re2.id_rider
      ) AS T
    )
);
```

Al utilizar subconsultas con `HAVING`, contemplamos todos los pilotos empatados a puntos en cada año y categoría.

Cuadro 4: Resultado Consulta 4: Pilotos campeones en Moto2 y Moto3 que no lo fueron en MotoGP

forename	**surname**
Alex	Márquez

Consulta 5: equipos con más títulos de MotoGP

Esta consulta calcula cuántos campeonatos de MotoGP ha logrado cada equipo gracias a sus pilotos. Primero identificamos al campeón o campeones de cada año y luego contamos cuántas veces ese piloto estuvo inscrito con cada equipo. Si un piloto cambia de equipo en una misma temporada, consideramos que el título se reparte entre todos los equipos con los que compitió. Por ese motivo usamos `SELECT DISTINCT` sobre el par `(año, id_team)`, de este modo cada equipo se cuenta una sola vez por temporada.

```sql
SELECT te.name, COUNT(*) AS num_campeones
FROM teams te
  INNER JOIN (
    SELECT DISTINCT ra.year, re.id_team
    FROM results re
      INNER JOIN races ra ON ra.year = re.year
        AND ra.sequence = re.sequence
        AND ra.category = re.category
    WHERE ra.category = 'MotoGP'
      AND (re.id_rider, ra.year) IN (
        SELECT re2.id_rider, ra2.year
        FROM results re2
          INNER JOIN races ra2 ON ra2.year = re2.year
            AND ra2.sequence = re2.sequence
            AND ra2.category = re2.category
        WHERE ra2.category = 'MotoGP'
        GROUP BY re2.id_rider, ra2.year
        HAVING SUM(re2.points) = (
          SELECT MAX(puntos_totales)
          FROM (
            SELECT SUM(re3.points) AS puntos_totales
            FROM results re3
              INNER JOIN races ra3 ON ra3.year = re3.year
                AND ra3.sequence = re3.sequence
                AND ra3.category = re3.category
            WHERE ra3.category = 'MotoGP'
              AND ra3.year = ra2.year
            GROUP BY re3.id_rider
          ) AS T
        )
      )
  ) C ON te.id_team = C.id_team
GROUP BY te.id_team, te.name
ORDER BY num_campeones DESC;
```

El uso de subconsultas con `MAX` en lugar de `ORDER BY ... LIMIT 1` nos garantiza que, en caso de empate, todos los pilotos campeones se tengan en cuenta. Al no utilizar `LIMIT`, evitamos descartar campeones empatados. La tabla resultante muestra los equipos ordenados de mayor a menor número de títulos.

Cuadro 5: Resultado Consulta 5: Equipos ordenados por número de campeonatos del mundo en MotoGP

name	num_campeones
Repsol Honda Team	8
Fiat Yamaha Team	3
Yamaha Factory Racing	2
Gauloises Yamaha Team	1
Ducati Marlboro Team	1
Ducati Team	1
Movistar Yamaha MotoGP	1
Team SUZUKI ECSTAR	1
Monster Energy Yamaha MotoGP	1

Consulta 6: circuitos sin vencedores locales

Queremos obtener el listado de circuitos donde nunca ha ganado un piloto cuya nacionalidad coincida con el país del trazado. Primero calculamos los circuitos que sí cuentan con al menos un vencedor local; luego, mediante una subconsulta con `NOT IN`, excluimos estos circuitos de la lista final. La normalización de los códigos de país a tres letras fue un paso esencial para que esta consulta diera resultados correctos.

```
SELECT ci.name, ci.country
FROM circuits ci
WHERE ci.id_circuit NOT IN (
    SELECT DISTINCT gp.id_circuit
    FROM grand_prix gp
      INNER JOIN races   ra ON ra.year     = gp.year
        AND ra.sequence = gp.sequence
      INNER JOIN results re ON re.year     = ra.year
        AND re.sequence = ra.sequence
        AND re.category = ra.category
      INNER JOIN riders  ri ON ri.id_rider = re.id_rider
      INNER JOIN circuits ci2 ON ci2.id_circuit = gp.id_circuit
    WHERE re.position = 1
      AND ri.nationality = ci2.country
)
ORDER BY ci.country, ci.name;
```

Al comparar `ri.nationality` y `ci.country` con el mismo formato, la consulta identifica correctamente aquellos circuitos en los que nunca ha habido un triunfo local.

Cuadro 6: Resultado Consulta 6: Circuitos donde ningún piloto local ha ganado en ninguna categoría

name	**country**
Termas De Río Hondo	ARG
Red Bull Ring – Spielberg	AUT
Nelson Piquet Circuit	BRA
Shanghai Circuit	CHN
Automotodrom Brno	CZE
Sepang International Circuit	MAL
TT Circuit Assen	NED
Estoril Circuit	POR
Losail International Circuit	QAT
Phakisa Freeway	RSA
Chang International Circuit	THA
Istanbul Circuit	TUR
Circuit Of The Americas	USA
Indianapolis Motor Speedway	USA

8. Conclusiones

A lo largo de esta práctica hemos comprobado los beneficios de aplicar el modelado relacional a un conjunto de datos desorganizado. Tras auditar la base original, identificamos las entidades clave y redactamos un modelo conceptual. Su traducción al modelo relacional nos permitió definir tablas en tercera forma normal con claves primarias y foráneas claras. La migración a SQLite y posterior exportación a MySQL se realizaron sin pérdida de información. Finalmente, las consultas diseñadas demuestran que el nuevo esquema facilita la extracción de información relevante como por ejemplo: identificar campeones, comparar países, encontrar pilotos polivalentes, analizar los resultados obtenidos de los equipos, etc. Consideramos que el trabajo realizado sienta una base sólida para futuros análisis y visualizaciones de los datos del mundial de motociclismo.